

AI-Powered Behavioral Anomaly Detection for Cybersecurity

Final Report

A system that learns normal access behaviour per user, service account and edge device, detects compromised credentials and intrusions in near real-time, classifies the anomaly type, and produces a calibrated, explainable risk score for a SOC analyst.

Detection	PR-AUC 0.938 · Recall @ 1% alert budget 0.896 · Macro-F1 0.861 (9 classes)
Calibration	Expected Calibration Error 0.0009
Latency	2.31 ms median, 3.39 ms p95 per event, single CPU core
Dataset	168,968 synthetic events · 260 entities · 45 days · 0.89% anomalies
Verification	643 automated tests , all passing · deterministic under seed 42

All figures measured on the held-out **test split** (38,875 events, 376 anomalies), which no model, threshold or calibrator ever saw. Reproduction commands in Section 10.

1. Approach

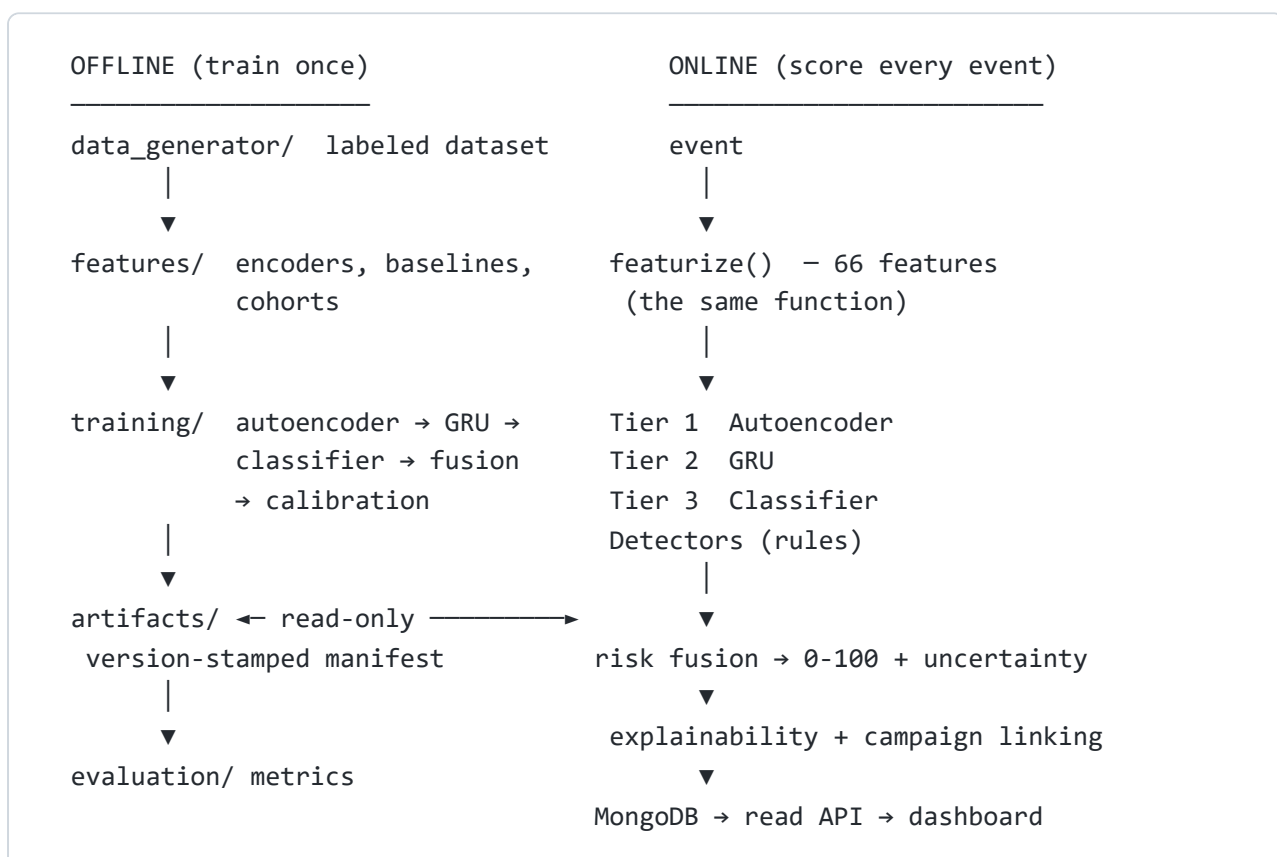
Signature-based security cannot detect what it has not already catalogued. We invert the problem:
learn *normal*, then flag deviation.

The core design principle is that **almost every feature is relative to the entity, not absolute.**

bytes_out = 4 MB means nothing. "*3.4 standard deviations above what this service account normally sends, to a resource it has never touched, from a country it has never used*" is a signal. This is why one model generalises across users, service accounts and devices that behave nothing alike.

Challenge	Response
Sequential behaviour	GRU language model over command sequences scores how surprising each next action is
Class imbalance (0.89%)	Accuracy never reported. PR-AUC and recall inside a 1%-of-events analyst budget are the headline metrics
Concept drift	Per-entity PSI with automatic re-profiling, so benign change is absorbed rather than alerted on
Explainability	SHAP, baseline diff, counterfactual, per-step sequence attribution and MITRE mapping on every alert
Cold start	New entities scored against a behavioral cohort prior, blended by a shrinkage weight that is itself a model feature

2. Architecture



Two decisions carry disproportionate weight.

One feature function, enforced. `featurize()` is called by training and by the online scorer;

batch processing calls it in a loop rather than reimplementing it vectorized. This is a deliberate

performance sacrifice — a second implementation would drift, and train/serve skew is the most common
silent ML failure: offline metrics stay excellent while production degrades. A test asserts batch and
event-by-event scoring produce **bit-identical** vectors.

No leakage by construction. The offline fit replays events in time order and featurizes each
against the profile built from *strictly earlier* events. A batch build would be faster and quietly
wrong — every event compared to a baseline containing itself. Labels are also stored in a **separate**
file from features, so feature code physically cannot read one.

All trained state persists as **JSON**, **never pickle** (a pickled estimator breaks on any library upgrade), under a manifest recording schema version, seed and git SHA. The scorer
refuses to run
against a mismatched schema.

3. Synthetic data

Real access logs with genuine credential compromise are not publicly available and would carry
personal data. Synthetic generation makes ground truth exact, guarantees every rare class in every
split, and — uniquely — allows benign concept drift to be *labelled benign*.

168,968 events, 260 entities, 45 days, 39,202 sessions, 0.89% anomalies. Six latent behavioral
cohorts (office staff, engineering, analytics, batch services, integration services, plant devices).
Regenerates byte-identically in ~21 s.

Eight attack behaviours, each with a distinct primary signal

Class	Events	Primary signal
low_and_slow_exfil	384	Sustained mildly-elevated transfers over many hours
lateral_movement	261	Fan-out across 5–13 resources outside the entity's cohort, with recon commands

Class	Events	Primary signal
credential_misuse	193	Valid auth, but several behavioral facets deviate at once
brute_force	189	7–22 failed auths against one entity in a ~4–9 min window
device_spoofing	138	Device fingerprint inconsistent with the entity's own history
insider_drift	132	Gradual self-directed escalation toward sensitive resources over days
credential_stuffing	114	One source IP against 9–26 entities, 1–3 attempts each
impossible_travel	91	Implied geo velocity far above 900 km/h

Without distinct signals the classifier would be guessing between overlapping classes and macro-F1 would be unreachable regardless of model quality.

Benign traffic is deliberately messy

If normal behaviour were clean, every attack would stand out and the metrics would be meaningless.

Measured: **23.2% of benign events are off-hours** (24/7 services, night batch), 0.34% are failed

authentications, 3% of sessions touch a first-time resource, 6% legitimately touch a sensitive

resource, and users occasionally travel. So "off-hours", "new resource" and "sensitive resource" are

all suspicious but never conclusive.

Difficulty was verified, not assumed

We fitted a naive model on raw per-event columns only — no baselines, no sequences, no windows — to establish the floor the real pipeline must beat:

	Naive floor	Full system
PR-AUC	0.706	0.938
Recall @ 1% budget	0.630	0.896

The classes the naive model fails on are exactly those needing the layered detectors (insider_drift 0.372, low_and_slow_exfil 0.448, lateral_movement 0.492). Three data leaks were found and removed this way, including an attacker OS string detectable from one event with no profiling at all.

Splits are by time, never random — train 99,047 / val 31,046 / test 38,875 — with anomaly density deliberately equalized (0.86% / 0.89% / 0.97%) because the budget threshold is tuned on validation and applied to test.

4. Detection stack

66 features per event (59 numeric + 7 categorical), computed in 0.12 ms: temporal likelihoods, geo velocity and distance-from-home, resource and auth likelihoods, sequence novelty, device novelty, volume z-scores against the entity's own distribution, windowed rates, and profile confidence. Unseen categories are *signal*, not error — every encoder reserves a code for "never seen" and reports a novelty flag.

The brief offered a choice for the baseline and sequence models. We built **one of each, on merit**.

Tier	Model	Configuration	Why this one
1	Autoencoder	66→32→16→32→66, ReLU, MSE, ≤120 epochs, early stop	One-Class SVM trains in $\sim O(n^2)$ — unusable at 100k events — and gives one opaque distance. A plain statistical profile is univariate: it can say "this hour is unusual" but never <i>"this combination never occurs"</i> , which is the zero-day case
2	GRU	Embed 32 → hidden 64, 1 layer, 54-token vocab, padding masked from loss	54 tokens over 20 steps gives self-attention nothing to exploit; a Transformer would overfit at higher cost. GRU has 3

Tier	Model	Configuration	Why this one
			gates vs LSTM's 4 — faster, equal or better on short sequences. Graph models adjacency, not order
3	LightGBM	9-class, 31 leaves, lr 0.05, balanced weights, per-class isotonic calibration	Strongest on tabular data with imbalance, and SHAP-friendly

The classifier consumes **68** inputs — the 66 features **plus** the autoencoder and GRU scores. This is deliberate stacking: it can use "multivariately odd" and "surprising ordering" as evidence without rediscovering either.

Deterministic detectors handle what is defined by physics rather than probability. A firing detector raises risk to a floor of 90 and overrides the predicted type.

Detector	Rule	Test result
brute_force	≥ 5 failed auths in 10 min	1.000 precision , 33 fired
impossible_travel	> 900 km/h over ≥ 500 km	0.789 anomaly precision, 19 fired

The 500 km floor matters: geolocation jitter over a few km with a short gap can imply an absurd velocity, so short hops never fire.

Risk fusion

Weights are found by grid search over the probability simplex on **validation**, optimising recall inside the alert budget: **autoencoder 0.00, GRU 0.30, classifier 0.70**. The fused score passes through isotonic calibration (24 knots) so a risk of 70 means roughly a 70% observed attack rate.

Two thresholds answer two different questions — `budget_threshold` 19.05 ("is this in the top 1% I can review?") and `alert_threshold` 60.0 ("is this an alert?"). Separating them lets us report recall at a *fixed analyst workload* rather than at a threshold chosen to flatter the result.

Every score carries an uncertainty band, **doubled for cold-start entities** — the honest counterpart to scoring against a cohort prior instead of an established baseline.

5. Explainability, cold start, drift, campaigns

Every alert is explained five ways

SHAP attributions, a field-by-field **baseline comparison**, a **counterfactual** ("*from India at 10:00 instead of Brazil at 02:00 this would score 18 instead of 91*"), **per-step sequence attribution**, and a **MITRE ATT&CK** mapping — all produced inside the scoring pipeline.

Two details matter. Explanations quote **raw values, not scaled ones**: "*0.4% of their logins are at this hour*" is actionable, " -2.3σ " is not. And the optional language model **narrates, never decides** — it cannot influence any score, and a deterministic template is always available, so the demo runs fully offline.

A useful property of the GRU: per-step negative log-likelihood is *simultaneously* the score and the attribution, so explanation and detection cannot disagree.

Cold start via behavioral cohorts

A global average is useless here — the "average" of a night-batch service account and a 9-to-5 analyst resembles neither. Instead entities are clustered by *how they behave* (KMeans, $k=6$, centroids stored as JSON). The clustering **rediscovered the generator's latent archetypes without being told they existed**: "*service_accounts active around the clock, including weekends*", "*users peaking at 11:00, on weekdays*", "*edge_devices around the clock*".

Resolution is hierarchical — entity → entity blended with cohort prior → cohort → global — and it is **shrinkage, not a switch**: an entity with 5 sessions carries some signal, and discarding it would be as wrong as trusting it fully. The blend weight $n/(n+12)$ is exposed as a feature so the models know how much to trust the comparison. **Cold-start recall 0.857** (32 entities, 21 anomalies).

Concept drift

Per-entity PSI over a 200-event rolling window, 10 bins, 50-sample minimum. Above 0.20, a benign-looking change triggers exponentially-weighted re-profiling instead of an alert. 36 entities (13.8%) undergo generated benign drift — schedule, location, device or resource — ramped over 6–12 days and labelled benign, so adaptation can be proven rather than asserted.

Benign drift flagged as drifting, no adaptation	52.8%
Benign drift flagged as drifting, with adaptation	13.5%
Abrupt injected shift	PSI 12.86 vs 0.20 threshold → correctly flagged

The system absorbs gradual benign change while still catching an abrupt one by a wide margin. That distinction is the whole point: `insider_drift` also changes gradually, but converges on sensitive resources and off-hours access.

Attack-story reconstruction

Real intrusions are stories, not isolated events. The dataset contains **18 multi-stage campaigns** across three kill chains (`brute_force` → `credential_misuse` → `lateral_movement` → `low_and_slow_exfil` , `credential_stuffing` → `impossible_travel` → `credential_misuse` , `device_spoofing` → `lateral_movement` → `low_and_slow_exfil`), each tagged with ground truth so reconstruction is *measured*, not merely shown. At serving time detections are linked into an ordered timeline the dashboard renders. **Stage-linking accuracy 1.000** — see the scope note in Section 9.

6. Analyst dashboard

React 18 + TypeScript + Vite behind nginx, backed by a read-only API (12 endpoints) so dashboard queries never compete with scoring for latency.

Page	Shows
Overview	Alert counts by severity, anomaly-type distribution, recent high-risk activity
Alerts	Ranked detection queue with filters; opens the explanation drawer
Entity Explorer	Per-entity history, learned profile, all detections highest-risk first
Storyline	Reconstructed campaigns as an ordered kill-chain timeline
Model Performance	Live metrics, per-class breakdown, precision-at-k curve
Drift	Per-entity PSI state and adaptation events
System Health	Reachability of every service, mirroring <code>docker compose ps</code>

The **explanation drawer** is where the explainability work surfaces: risk gauge with uncertainty band, SHAP chart, counterfactual panel, highlighted command sequence, MITRE chips, and feedback buttons.

The feedback loop is real. A `false_positive` verdict applies a signed risk offset to that entity's future scoring. The offset is applied to the *decision*, not the model — so it is auditable, instantly reversible, needs no retraining, and cannot silently corrupt a trained model.

7. Results

Test split: 38,875 events, 376 anomalies (0.967% prevalence).

Metric	Result	Target	
PR-AUC	0.9380	≥ 0.90	PASS
Recall @ 1% alert budget	0.8963	≥ 0.80	PASS
Macro-F1 (9 classes)	0.8607	≥ 0.85	PASS
Calibration ECE	0.0009	≤ 0.05	PASS
Latency (median / p95)	2.31 / 3.39 ms	< 50 ms	PASS
ROC-AUC	0.9956	context only	—

What the 1% budget means operationally

A 1% budget is **389 events** an analyst reviews. Inside it: **337 of 376 attacks found, 52 false positives**, precision **0.866**. So reviewing the top 1% of ranked events surfaces roughly seven in eight of all attacks in the period.

Per-class classification

Class	Precision	Recall	F1	Support
normal	0.9996	0.9963	0.9980	38,499
lateral_movement	1.000	1.000	1.000	63
brute_force	1.000	1.000	1.000	45
low_and_slow_exfil	1.000	1.000	1.000	96
credential_misuse	1.000	0.974	0.987	39
credential_stuffing	1.000	0.889	0.941	36
insider_drift	0.833	0.698	0.759	43
impossible_travel	0.607	0.944	0.739	18
device_spoofing	0.196	0.889	0.322	36

`lateral_movement` and `low_and_slow_exfil` reach F1 1.000 — the two classes the naive baseline was worst at, which is direct evidence the sequence and windowed features earn their place. `device_spoofing` is the clear weak point (Section 9).

643 automated tests cover the domain contracts, generator statistics, feature parity, every model tier, fusion, calibration, explainability, drift, campaign linking, serving, API and evaluation.

8. Assumptions

1. **One organization across four countries.** A genuinely global company would weaken every geo signal.
2. **Sessions are well-formed;** real telemetry has orphaned and truncated sessions.
3. **Device fingerprints are stable**, changing only via drift or spoofing; real ones churn from VPNs and NAT.
4. **Geolocation is accurate** to city level; real IP geolocation is often wrong.

5. **A small closed command vocabulary** (~54 tokens); real command telemetry has a long tail of thousands.
6. **No label noise, and no adversarial adaptation** — attackers do not learn from being detected or deliberately mimic a specific victim's baseline.
7. **An entity's recent past predicts its near future, and behavioural cohorts transfer** as a prior for new entities. These are the premises of the whole approach.
8. **A 1%-of-events review budget** reflects realistic analyst capacity.

9. Known limitations

We would rather state these than have them found.

Cold-start uplift measures as exactly zero. Recall is 0.857 both with and without cohort priors.

The target passes, but the *mechanism* is not demonstrated to be the reason — the ablation appears to disable priors in a way the rest of the entity-relative feature vector compensates for. The cohort machinery is sound and the clustering recovers real structure, but we cannot claim a measured uplift on this dataset, so we do not.

device_spoofing precision is 0.196 — 131 normal events misclassified, the main drag on macro-F1.

The cause is instructive: we deliberately made spoofing detectable *only* from per-entity device history (an unfamiliar MAC on an ordinary corporate OS) after removing the globally-obvious variant as a data leak. That made the class realistic and hard, and the model now over-triggers on benign device churn. This is the clearest target for further work.

The 1.000 campaign metric has narrow scope. It measures whether flagged events belonging to a true campaign were grouped with their campaign-mates. It does *not* penalise over-segmentation — 55 groups were formed against 4 ground-truth campaigns in the split, because non-campaign anomalies also group — and detection misses are not charged to it.

The autoencoder carries zero linear fusion weight. It still feeds the classifier as an input and remains the source of per-feature reconstruction explanations, but it does not

independently drive ranking here. Honest reading: on data this well-featurized the supervised classifier subsumes its detection contribution. Its value would be greater with little or no labelled attack data — the realistic case, and why we keep it.

Per-class metrics rest on few independent incidents (3–6 for some classes). A regeneration with a different seed can move an individual class by ± 0.2 ; only aggregate metrics are stable. This is the most important caveat when reading the per-class table.

The `impossible_travel` detector's type precision is 0.263. It is reliably right that something is wrong (0.789 anomaly precision) but a poor type oracle, because impossible travel often co-occurs with other campaign stages.

The data is synthetic. The generator and detector share assumptions — the fundamental limit of any synthetic evaluation. These numbers are an upper bound on signal cleanliness, not a production guarantee. Related: `low_and_slow_exfil` spans hours rather than the days-to-weeks a real campaign would, and drift is modelled as linear where real change is bursty.

Streaming is implemented but not wired end to end. A Redis Streams consumer and a WebSocket endpoint exist, but no compose service runs the consumer and the dashboard fetches on navigation. The scoring path is stateless and horizontally scalable; the gap is transport wiring, not architecture.

10. Reproducing every number

```
python -m venv .venv ; .\.venv\Scripts\Activate.ps1 ; pip install -r requirements.t
python -m data_generator.generate --seed 42      # dataset, ~21s, byte-identical
python -m training.build_artifacts                # train all models -> artifacts/
docker compose up -d --build                     # mongodb, redis, scorer, api, dash
python -m serving.replay --split test --mongo --fresh
python -m evaluation.evaluate                     # -> artifacts/metrics.json
python -m evaluation.coldstart_experiment
python -m evaluation.campaign_experiment
```

```
python -m evaluation.drift_experiment
python -m evaluation.report          # -> REPORT.md
nvtest                               # 643 tests
```

Dashboard :8080 · read API :8000 · scorer :8100.

One seed (42) drives random , NumPy, PyTorch and LightGBM, with independent child streams per generation stage so changing the attack mix does not reshuffle benign traffic. The scoring service is the only component with a write path and requires a bearer token; the default is a development placeholder and must be changed before network exposure.

11. Deliverables and evaluation criteria

#	Deliverable	Implementation
1	Synthetic data generator + assumptions + attack taxonomy	data_generator/ + TAXONOMY.md
2	Baseline behavioural profiling model	models/baseline.py — autoencoder
3	Sequence-aware detection model	models/sequence.py — GRU
4	Anomaly classification model	models/classifier.py — calibrated LightGBM
5	Explainability layer	explainability/ — SHAP, counterfactual, sequence, MITRE
6	Analyst dashboard: ranked alerts, risk scores, entity history	frontend/ — 7 pages
7	Final report: assumptions, metrics, limitations	this document

Criterion	Evidence
Detection accuracy on imbalanced data	PR-AUC 0.938 at 0.97% prevalence; accuracy deliberately not reported
Anomaly-type classification	Macro-F1 0.861 over 9 classes; three at F1 1.000

Criterion	Evidence
Low false positives (top 1% budget)	Recall 0.896 within 389 events at precision 0.866
Explainability	SHAP + baseline diff + counterfactual + sequence attribution + MITRE per alert
Cold-start handling	Cohort priors with shrinkage; recall 0.857 (caveat §9)
Concept drift	PSI + adaptive re-profiling: benign false flagging 52.8% → 13.5%, abrupt shift still caught
Scalable design	Stateless scorer, 2.31 ms median, containerised (wiring gap §9)
Report quality	This document; every figure reproducible via §10

Artifact schema 1.0.0 · seed 42 · all figures on the held-out test split.

All changes saved